Node.js — Day 1 to Day 3 Complete Notes

Goal: Build a strong Node.js/backend foundation so you can understand and build real backends, connect them to your Vue/frontend knowledge, and eventually build a complete full-stack project.

Note: These notes summarize what we actually covered in Days 1–3 and are designed to be copied into Notion for future revision.

Day 1 — Node.js Fundamentals

What is Node.js?

Node.js is a JavaScript runtime that lets JavaScript run outside the browser. It is commonly used for backend applications, APIs, servers and command-line programs.

Remember: JavaScript is the programming language; Node.js is a runtime/environment for executing JavaScript outside the browser.

Why Node.js?

Node.js lets you use JavaScript on both frontend and backend. It is particularly useful for applications involving many I/O operations such as database access, network requests and file operations.

Node.js is not a framework

Node.js is the runtime. Express.js is a framework we will learn later to make HTTP servers and APIs easier to build.

Running JavaScript with Node

```
console.log("Hello Node.js");
```

Save the file and run it in the terminal with **node filename.js**.

Mental model

Think of Node.js as the environment in which your server-side JavaScript runs. Later we will use it with Express, databases, authentication and APIs.

Day 2 — Synchronous vs Asynchronous JavaScript

Synchronous execution

Synchronous code executes in order. The next statement waits until the current statement finishes.

```
console.log("A");
console.log("B");
console.log("C");
```

Output: A, B, C.

Asynchronous execution

Asynchronous operations allow JavaScript to continue with other work instead of waiting for the asynchronous operation to finish.

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);
```

```
console.log("C");
```

Output: A, C, B.

setTimeout(..., 0)

0ms does not mean immediately. It schedules the callback. The callback still waits until the current synchronous work is finished and Node.js can process it.

Callbacks

A callback is a function supplied to another function so it can be executed later, often when an asynchronous operation is ready.

```
setTimeout(() => {
  console.log("Done");
}, 1000);
```

Timer ordering

When similar timers are ready, their callbacks are generally processed in the order they were scheduled. A longer delay makes a timer eligible later.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
setTimeout(() => console.log("3"), 0);
console.log("4");
```

Output: 1, 4, 2, 3.

Day 2 rule: Synchronous code runs first. Asynchronous callbacks wait until the current synchronous work is finished and they can be processed.

Day 3 — Event Loop

Call Stack

The Call Stack is where JavaScript keeps track of the functions currently being executed. Synchronous function calls are placed on the stack and removed when they finish.

```
function test() {
  console.log("C");
}

test();
```

Asynchronous work

When Node.js encounters an asynchronous operation such as a timer, JavaScript does not simply stop and wait. The runtime handles the asynchronous operation while JavaScript continues with other synchronous work.

```
console.log("Start");
setTimeout(() => {
  console.log("Hello");
}, 2000);
console.log("End");
```

The important order is Start, End, then Hello after the timer is ready and its callback can be processed.

Event Loop — beginner mental model

```
JavaScript
    ↓
Call Stack
```

```
    ↓
Node.js handles asynchronous work
    ↓
Callback becomes ready
    ↓
Event Loop
    ↓
Call Stack
```

The Event Loop helps coordinate when callbacks that are ready can be processed by JavaScript. This allows Node.js to keep moving instead of blocking JavaScript while waiting for every asynchronous operation.

I/O

I/O means Input/Output. Examples include reading/writing files, database operations, network requests and other communication with resources outside immediate JavaScript execution.

Node.js and 'single-threaded'

For our beginner mental model, JavaScript execution in Node.js primarily runs on a main thread. Node.js also uses its runtime and underlying system mechanisms to handle asynchronous work. The key idea is that JavaScript does not need to block on every I/O operation.

Important rules

1. Synchronous code runs first.

2. setTimeout(..., 0) does not mean immediate execution.

3. Timers become eligible according to their delays; similar ready callbacks are generally processed in scheduling order.

4. The Event Loop coordinates when ready callbacks can be processed.

5. I/O is a major reason asynchronous Node.js is useful for backend applications.

Day 1–3 Quick Revision

Node.js: JavaScript runtime outside the browser.

Synchronous: Executes in order; the next statement waits for the current work.

Asynchronous: Allows execution to continue while an asynchronous operation is being handled.

Callback: A function that can execute later when an operation is ready.

Call Stack: Tracks currently executing JavaScript functions.

Event Loop: Helps coordinate ready asynchronous callbacks with JavaScript execution.

I/O: Input/Output such as database, file and network operations.

Core mental model: synchronous work runs first; asynchronous callbacks wait until the current synchronous work is finished and the runtime can process them.

Day 4 — Starting Exercise

We will begin Day 4 by solving the exercise from Day 3 about a heavy synchronous loop and a 0ms timer. This introduces the important concept of **blocking the Event Loop**, then we will continue to the next topic.